

TP 3 : Métriques d'évaluation d'un classifieur supervisé et jeux de données d'entraînement et de test

Le but de ce TP va être :

- D'écrire des fonctions permettant de calculer l'accuracy et les autres métriques vues en cours pour évaluer les performances d'un classifieur
- De voir comment partager un jeu de données en plusieurs portions afin d'effectuer l'apprentissage et l'évaluation du classifieur

Pour simplifier nous allons travailler avec un classifieur binaire simple (le modèle de régression logistique) et n'utiliser que certaines de nos variables pour prédire la survie au naufrage.

1. Jeu de données

On charge le jeu de données obtenu en fin de TP 2 :

```
library(dplyr)
library(tidyverse)
# Assurez-vous que le fichier csv est dans le répertoire de travail
data = read.csv("titanic_pre_processed.csv", stringsAsFactors = TRUE)
y = data$Survived # notre target
X = data %>% select(Fare, Fam) # pour simplifier on n'utilise que deux de nos variables explicatives
```

2. Un classifieur simple

Nous utiliserons le classifieur binaire implémenté ci-dessous :

```
myClassifieur = function (X_train, y_train, X_test, threshold = 0.5){
  data_train = cbind(y_train, X_train)
  mod = glm(y_train ~ ., data = data_train, family = binomial(link = "logit"))
  # Prédiction : si proba > threshold alors 1, sinon 0
  y_pred = 0 + 1 * (predict.glm(mod, newdata = X_test, type = "response") > threshold)
  return(y_pred)
}
```

Quelle est la valeur seuil du classifieur par défaut ?

Réponse : 0.5

Par exemple, on peut entraîner le classifieur sur les données complètes, puis prédire la target des 10 premières observations du jeu de données, pour une valeur seuil de 0.5 et pour une valeur seuil de 0.3.

```
# Prédiction avec seuil 0.5
y_pred_05 = myClassifieur(X_train = X, y_train = y,
                          X_test = X[1:10,], threshold = 0.5)

# Prédiction avec seuil 0.3
y_pred_03 = myClassifieur(X_train = X, y_train = y,
                          X_test = X[1:10,], threshold = 0.3)

# Affichage pour comparaison
rbind(Reel = y[1:10], Pred_0.5 = y_pred_05, Pred_0.3 = y_pred_03)
```

```
##           1 2 3 4 5 6 7 8 9 10
## Reel      0 1 1 1 0 0 0 0 1 1
## Pred_0.5  0 1 0 0 0 0 0 0 0 0
## Pred_0.3  1 1 1 1 1 1 1 0 0 1
```

Que remarque-t-on ?

Réponse : En abaissant le seuil à 0.3, on prédit plus facilement la classe positive (1). Cela augmente le nombre de “Survécu” prédits, ce qui risque d’augmenter les faux positifs.

3. Ensembles d’apprentissage et de test - méthode holdout

Que font les lignes de code ci-dessous ?

```
n      = nrow(data)
inTrain = sample(1:n, size = round(0.8 * n), replace = FALSE)
X_train = X[inTrain,]
y_train = y[inTrain]
X_test  = X[-inTrain,]
y_test  = y[-inTrain]
```

Réponse : Ces lignes sélectionnent aléatoirement 80% des indices du jeu de données pour constituer l’ensemble d’apprentissage (**train**). Les 20% restants (exclus via **-inTrain**) constituent l’ensemble de test (**test**).

Écrire une fonction nommée **holdOut** qui implémente la méthode du holdout.

```
holdOut = function(X, y, split = 0.8){
  n      = nrow(X)
  # Tirage aléatoire des indices
  inTrain = sample(1:n, size = round(split * n), replace = FALSE)

  # Création des sous-ensembles
  X_train = X[inTrain,]
  y_train = y[inTrain]
  X_test  = X[-inTrain,]
  y_test  = y[-inTrain]

  return(list(X_train = X_train, y_train = y_train,
              X_test = X_test, y_test = y_test))
}
```

Que fait la ligne de code ci-dessous ? À quoi cela peut-il servir ?

```
set.seed(1234)
```

Réponse : Elle fixe la “graine” du générateur de nombres aléatoires. Cela garantit que le partage train/test sera identique à chaque exécution du code (reproductibilité).

Utiliser la fonction **holdOut** sur le jeu de données **data** avec un split train/test de 80/20.

```
data_split = holdOut(X = X, y = y, split = 0.8)
```

Utiliser **myClassifier** pour prédire la target des individus dans le jeu de données de test.

```
y_pred = myClassifier(X_train = data_split$X_train, y_train = data_split$y_train,
                      X_test = data_split$X_test, threshold = 0.5)
```

4. Métriques d’évaluation

4.1. Accuracy, recall, precision, F1score

Écrire 3 fonctions nommées **myAccuracy**, **myRecall**, et **myPrecision**.

```
myAccuracy = function(y_true, y_pred){
  # Proportion de prédictions correctes
```

```

    return(mean(y_true == y_pred))
}

myRecall = function(y_true, y_pred){
  # Vrais Positifs / (Vrais Positifs + Faux Négatifs)
  # C'est la capacité à trouver tous les "1"
  TP = sum((y_true == 1) & (y_pred == 1))
  TotalPositifs = sum(y_true == 1)
  return(TP / TotalPositifs)
}

myPrecision = function(y_true, y_pred){
  # Vrais Positifs / (Vrais Positifs + Faux Positifs)
  # C'est la fiabilité quand on prédit "1"
  TP = sum((y_true == 1) & (y_pred == 1))
  TotalPredPositifs = sum(y_pred == 1)

  # Gestion du cas où on ne prédit aucun positif
  if(TotalPredPositifs == 0) return(0)

  return(TP / TotalPredPositifs)
}

# Tests des fonctions
myAccuracy(data_split$y_test, y_pred)

```

```
## [1] 0.6685714
```

```
myRecall(data_split$y_test, y_pred)
```

```
## [1] 0.2153846
```

```
myPrecision(data_split$y_test, y_pred)
```

```
## [1] 0.6666667
```

Installer puis charger le package Metrics.

```

# install.packages("Metrics")
library(Metrics)
# Détacher caret s'il est chargé pour éviter les conflits de noms
if("package:caret" %in% search()) detach("package:caret", unload = TRUE)

accuracy(data_split$y_test, y_pred)

```

```
## [1] 0.6685714
```

```
recall(data_split$y_test, y_pred)
```

```
## [1] 0.2153846
```

```
precision(data_split$y_test, y_pred)
```

```
## [1] 0.6666667
```

Le package caret propose aussi des fonctions permettant de calculer certaines de ces métriques.

Réponse : La fonction `recall` de `caret` permet de spécifier quelle classe est la classe "positive" (argument `mode` ou `relevant`), ce qui est utile si les classes ne sont pas 0 et 1. De plus, il y a un conflit de noms :

charger `caret` après `Metrics` masque les fonctions de `Metrics`.

Le package `caret` permet aussi d'obtenir la matrice de confusion avec la commande `confusionMatrix`.

```
library(caret)
# On convertit en facteur pour caret
cm = confusionMatrix(data = factor(y_pred), reference = factor(data_split$y_test),
                     positive = "1")
cm$table
```

```
##           Reference
## Prediction    0    1
##           0 103  51
##           1   7  14
```

En utilisant le package `caret`, donner la valeur de la spécificité et de la balanced accuracy. Commenter.

```
metrics = cm$byClass
metrics["Specificity"]
```

```
## Specificity
##  0.9363636
```

```
metrics["Balanced Accuracy"]
```

```
## Balanced Accuracy
##      0.5758741
```

Commentaire : La spécificité (taux de vrais négatifs) est souvent plus élevée que le recall. La Balanced Accuracy fait la moyenne arithmétique de la sensibilité (Recall) et de la spécificité, donnant une vision plus juste si les classes sont déséquilibrées.

Prédire à nouveau la target mais pour différentes valeurs de `threshold` dans la fonction `myClassifier`.

```
# Seuil plus élevé (0.7)
y_pred_high = myClassifier(X_train = data_split$X_train, y_train = data_split$y_train,
                          X_test = data_split$X_test, threshold = 0.7)

myRecall(data_split$y_test, y_pred_high)
```

```
## [1] 0.1076923
```

```
myPrecision(data_split$y_test, y_pred_high)
```

```
## [1] 0.7777778
```

Réponse : Avec un seuil de 0.7, on prédit moins souvent "Survivant". Le Recall baisse (on rate des survivants), mais la Précision augmente (ceux qu'on identifie sont vraiment des survivants).

Que font les lignes de code ci-dessous ? (Comparaison Train vs Test)

```
# Prédiction sur le jeu d'ENTRAÎNEMENT
y_pred_train = myClassifier(X_train = data_split$X_train, y_train = data_split$y_train,
                          X_test = data_split$X_train, threshold = 0.5)

cat("Sur le TRAIN :\n")
```

```
## Sur le TRAIN :
```

```
cat("Recall:", myRecall(data_split$y_train, y_pred_train), "\n")
```

```
## Recall: 0.2608696
```

```
cat("Precision:", myPrecision(data_split$y_train, y_pred_train), "\n")
```

```
## Precision: 0.72
```

```
cat("\nSur le TEST :\n")
```

```
##
```

```
## Sur le TEST :
```

```
cat("Recall:", myRecall(data_split$y_test, y_pred), "\n")
```

```
## Recall: 0.2153846
```

```
cat("Precision:", myPrecision(data_split$y_test, y_pred), "\n")
```

```
## Precision: 0.6666667
```

Réponse : On compare les performances sur les données connues (train) et inconnues (test). Les résultats sur le Train sont généralement meilleurs (optimistes), ce qui peut indiquer un sur-apprentissage (overfitting).

4.2. AUC

Modifier `myClassifieur` pour créer une fonction nommée `myClassifieur2` qui retourne la probabilité.

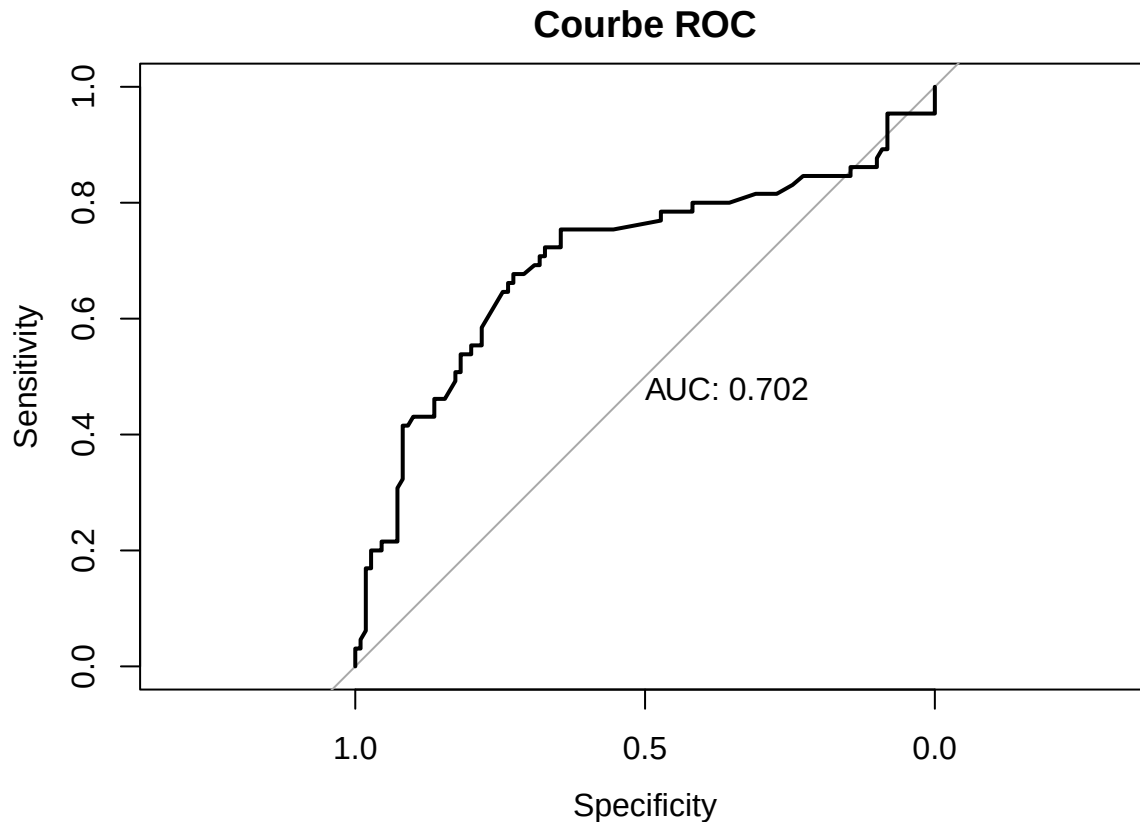
```
myClassifieur2 = function (X_train, y_train, X_test){
  data_train = cbind(y_train, X_train)
  mod = glm(y_train ~ ., data = data_train, family = binomial(link = "logit"))
  # On retourne directement la probabilité prédite
  y_prob = predict.glm(mod, newdata = X_test, type = "response")
  return(y_prob)
}
```

Prédire la probabilité et tracer la courbe ROC.

```
# install.packages("pROC")
library(pROC)

# Prédiction des probabilités
y_prob = myClassifieur2(X_train = data_split$X_train, y_train = data_split$y_train,
                        X_test = data_split$X_test)

# Courbe ROC et AUC
roc_obj = roc(response = data_split$y_test, predictor = y_prob)
plot(roc_obj, print.auc = TRUE, main = "Courbe ROC")
```



Commentaire : L'axe Y est la Sensibilité (Recall), l'axe X est la Spécificité. Une AUC (Area Under Curve) proche de 1 indique un classifieur parfait. 0.5 équivaut au hasard.

5. Ensembles d'apprentissage et de test - k-fold cross validation

Que font les lignes de code ci-dessous ?

```
set.seed(1234)
nfolds = 10
folds = NULL
# Création manuelle des folds
for(i in 1:n){
  folds = c(folds, sample(1:nfolds, 1, replace = FALSE))
}
```

Réponse : Cela assigne aléatoirement chaque observation à l'un des 10 groupes (folds) pour la validation croisée.

Que fait la fonction CV donnée ci-dessous ?

```
CV = function(X, y, folds){
  nfolds      = max(folds)
  accuracy_k  = numeric(nfolds)
  recall_k    = numeric(nfolds)

  for(k in 1:nfolds){
    # On identifie les indices appartenant au fold k
    inFold     = which(folds == k)
```

```

# Séparation Train (tout sauf k) / Test (seulement k)
X_train = X[-inFold,]
X_test  = X[inFold,]
y_train = y[-inFold]
y_test  = y[inFold]

# Entraînement et prédiction
y_test_pred = myClassifier(X_train = X_train, y_train = y_train,
                           X_test = X_test, threshold = 0.5)

# Stockage des métriques
accuracy_k[k] = myAccuracy(y_test, y_test_pred)
recall_k[k]   = myRecall(y_test, y_test_pred)
}
return(list(accuracy_k = accuracy_k, recall_k = recall_k ))
}

```

Utiliser de manière répétée les différentes portions du jeu de données (Cross Validation).

```

# Exécution de la validation croisée
results_cv = CV(X, y, folds)

# Calcul des moyennes
mean_accuracy = mean(results_cv$accuracy_k)
mean_recall   = mean(results_cv$recall_k)

cat("Accuracy moyenne sur 10 folds :", mean_accuracy, "\n")

## Accuracy moyenne sur 10 folds : 0.6680665

cat("Recall moyen sur 10 folds :", mean_recall, "\n")

## Recall moyen sur 10 folds : 0.2555457

```

6. Prochaines étapes

On pourra utiliser nos fonctions `holdOut` et `CV`, ou celles du package `caret`, pour partager le jeu de données en train et test.